

# 昨年度作品からの成長

## ～『Slime's Sky Travel』から『Slime's Space Travel』へ～

2026. 9. 23

日下 拓海

### 目次

1. 本資料について
2. 描画・3D 表現の成長
3. 移動・物理処理の成長
4. 開発環境・設計の成長
5. 遊びの幅と遊びやすさの両立
6. プレイヤー向けステージ作成機能
7. 次回作でさらに実現したいこと
8. まとめ

### 1. 本資料について

#### 1-1. 目的

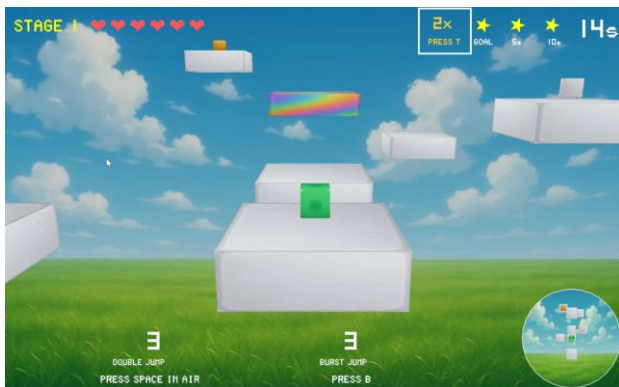
昨年度に制作した『Slime's Sky Travel』と、本年度に制作した『Slime's Space Travel』を比較し、この1年間で技術・設計・ゲーム体験への考え方がどのように発展したかを整理します。単なる機能や使用技術の比較ではなく、前作で見つかった課題を本年度作でどう改善し、遊びやすさや分かりやすさへ結び付けたかを中心にまとめます。

#### 1-2. 昨年度から本年度へ

昨年度は、C++ / OpenGL を用いて 3D ゲームを一作完成させ、描画、入力、衝突判定、UI、ステージ管理などの基礎的な仕組みを実装しました。その経験から、描画方式、衝突判定、クラス設計、ステージ調整などに改善点が見つかりました。また、Easy Mode を用意するなど、当時からゲームに慣れていない人への遊びやすさも意識していました。

本年度は、これらの経験を基に技術基盤を発展させながら、戦闘や2人プレイなど遊びの幅を広げました。その一方で、機能の増加によって操作や理解のハードルまで高くないようにし、「遊びの幅を広げても、遊ぶハードルは上げない」ことを一つの軸として開発しました。

昨年度作品



今年度作品



## 2. 描画・3D 表現の成長

### 2-1. 固定機能パイプラインからプログラマブルパイプラインへ

昨年度作品では、glBegin / glVertex3f / glMatrixMode などを利用した OpenGL の固定機能・即時モードを中心に描画し、プレイヤーや足場も立方体などのプリミティブへテクスチャを貼って表現していました。

本年度作品では、VAO / VBO / EBO と GLSL Shader を用いるプログラマブルパイプラインへ移行しました。頂点データや描画処理を自分で管理する構成へ変更し、3D モデル、アニメーション、Shadow Mapなどを追加できる描画基盤へ発展させました。

### 2-2. プリミティブ表現から 3D モデル・スケルタルアニメーションへ

昨年度作品では、キャラクターの状態や行動をアニメーションで表現する仕組みはありませんでした。本年度作品では Assimp を用いて FBX モデル、Bone Weight、Animation Clip を読み込み、キーフレーム補間と Quaternion SLERP によるスケルタルアニメーションを実装しました。これにより、待機、移動、ジャンプ、攻撃、回避などをキャラクター自身の動きとして表現できるようにしました。

見た目を豊かにするだけでなく、プレイヤーが入力した結果や現在の行動を視覚的に理解しやすくすることも重視しています。Shadow Map では接地位置や高低差を伝えやすくし、Post Process では攻撃や演出の視覚的なフィードバックを強めることを狙いました。

昨年度作品



今年度作品



### 3. 移動・物理処理の成長

#### 3-1. Y 軸固定の移動から球・楕円体の 360°移動へ

昨年度作品では、重力方向をワールド座標の-Y 方向に固定し、Y 軸を常に上として扱っていました。本年度作品では惑星の表・側面・裏面までを一つのステージとして利用するため、位置に応じて上方向や重力方向を変化させる必要がありました。

そこで Actor ごとに上方向を表す **upVec** を持たせ、接地面や引力対象の惑星に応じて姿勢と重力方向を変化させる仕組みを実装しました。楕円体ではプレイヤー位置から表面上の最近点と外向き法線を求め、空中で引力対象が切り替わる場合には **upVec** を補間して急激な姿勢変化を抑えています。これにより、惑星全体を利用した 360°アクションを実現しながら、姿勢や重力方向の変化による違和感を抑えることを目指しました。

昨年度作品



本年度作品



#### 3-2. AABB 中心の判定から Ray Test・Sweep Test へ

昨年度作品の衝突判定は、プレイヤーと足場の最小・最大座標を比較する AABB 判定が中心でした。本年度作品では、球体や楕円体、斜面、突き出した足場など複雑な地形を扱うため、Bullet Physics を利用した接地・衝突判定へ発展させました。

接地判定では中央・前後左右から Ray Test を行い、取得した地面法線から **upVec** を決定しています。また移動時には Sweep Test で移動経路上の衝突を確認し、壁へ衝突した場合は残りの移動量から衝突法線方向の成分を除いて壁沿いに移動させています。360°移動を追加しながら、引っ掛かりやすり抜けを抑え、入力に対して自然に動く操作感を目指しました。

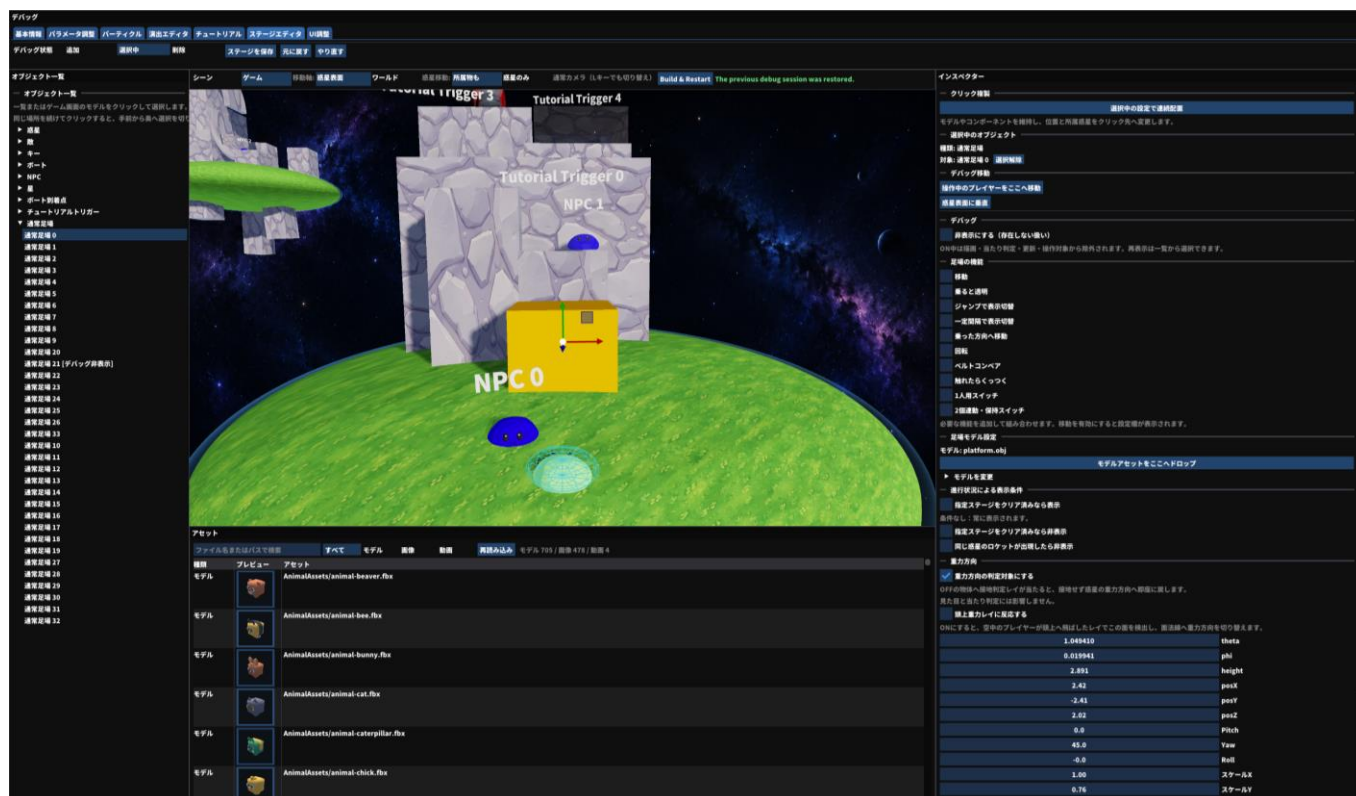
### 4. 開発環境・設計の成長

#### 4-1. 数値編集・ホットリロードから画面上での直接編集へ

昨年度作品でも、Stage・Texture・Audio・UI Config を実行中に再読み込みするホットリロードを用意し、再起動の手間を減らしていました。一方、足場や UI の配置調整では、設定値を変更してからゲーム画面で結果を確認する往復が必要でした。

本年度作品では、ゲーム画面を Framebuffer へ描画して Debug Editor 内に表示し、ImGuizmo を用いて Actor を画面上から直接移動・回転・拡大縮小できる 3D ステージエディタを実装しました。UI も実際のゲーム画面上で選択し、配置変更や複製、削除、Undo、保存を行えます。

調整の反復を短くすることで、足場の間隔や敵の配置、UI の位置などを繰り返し試し、遊びやすさや分かりやすさを追求しやすい制作環境へ発展させました。



ゲーム画面上で Actor を直接編集できる 3D ステージエディタ

## 4-2. 機能追加に耐えられる責務分割

昨年度作品では、GameState が Player、Camera、Item、Skill、Replay、UI など多くの状態を保持し、Platform にも複数の役割が集まっていました。

本年度作品では機能が大きく増えるため、Player の処理を PlayerInput、PlayerMovement、PlayerGrounding、PlayerCombat などへ分割しました。Enemy についても移動、戦闘、状態管理、行動 AI を分離し、ゲーム全体も System や Controller へ責務を分けています。

クラスを細かくすること自体ではなく、操作感なら Movement、戦闘なら Combat というように変更箇所と影響範囲を判断しやすくし、機能を増やした後も各要素を独立して調整・改善できる構造を目指しました。

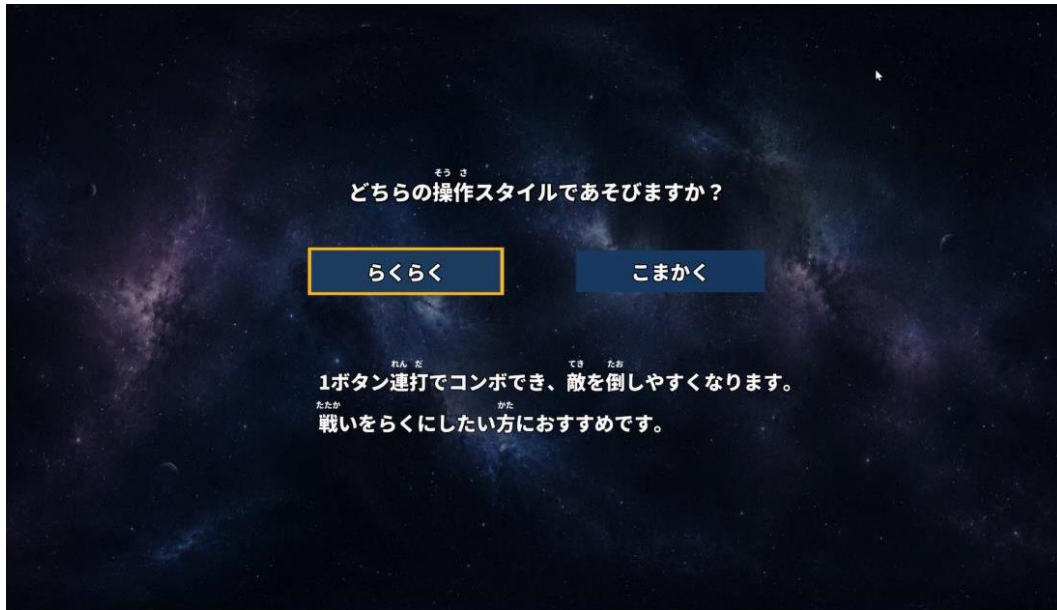
## 5. 遊びの幅と遊びやすさの両立

### 5-1. 戦闘追加と「らくらく／こまかく」スタイル

昨年度作品には戦闘がなく、移動とジャンプを中心に遊ぶ構成でした。一方で Easy Mode を用意し、落下してもライフを失わず最後に乗った足場から再開できる、制限時間を延長する、空中でもう一度ジャンプできるなど、アクションが苦手な人でも進めやすくする工夫を行っていました。

本年度作品では、コンボ、強攻撃、ガード破壊、打ち上げ、空中攻撃、ため攻撃などを組み合わせられる戦闘を追加しました。一方で、選べる行動が増えるほど操作も複雑になります。

そこで、技を自分で使い分ける「こまかくスタイル」に加え、同じ攻撃ボタンを連続して入力することでコンボへつなげやすい「らくらくスタイル」を実装しました。前作の「失敗したときの負担を減らす支援」から、「複雑な遊びそのものへ参加しやすくする支援」へと考え方を発展させています。



プレイヤーに合わせて操作方法を選べる「らくらく／こまかく」スタイル

## 5-2. ゲームパッド・2人プレイによる参加方法の拡張

昨年度作品はキーボード操作を中心とした1人用の構成でした。本年度作品ではゲームパッドにも対応し、プレイヤーごとに入力を扱える構成として1人プレイと2人プレイの両方に対応しました。

1人では分身した2体のスライムを切り替えて操作し、2人ではそれぞれが1体ずつ担当します。入力方法の選択肢を増やすだけでなく、家族や友人が同じステージ攻略へ参加できるようにし、遊びへの参加方法そのものを広げました。

## 5-3. 英語 Bitmap Font から日本語＋自動ルビへ

昨年度作品では、自作 Bitmap Font で英数字や記号を描画し、説明も英語表記が中心でした。本年度作品では SDL\_ttf を利用した UTF-8 日本語描画へ変更し、Windows 版では JapanesePhoneticAnalyzer から漢字の読みを取得して自動でルビを表示する仕組みも実装しました。

操作を簡単にするだけでなく、説明を読んで理解できることも遊び始めやすさの一部だと考え、特に子どもが漢字を読めないことで進め方を理解しにくくなる状況を減らすことを狙っています。

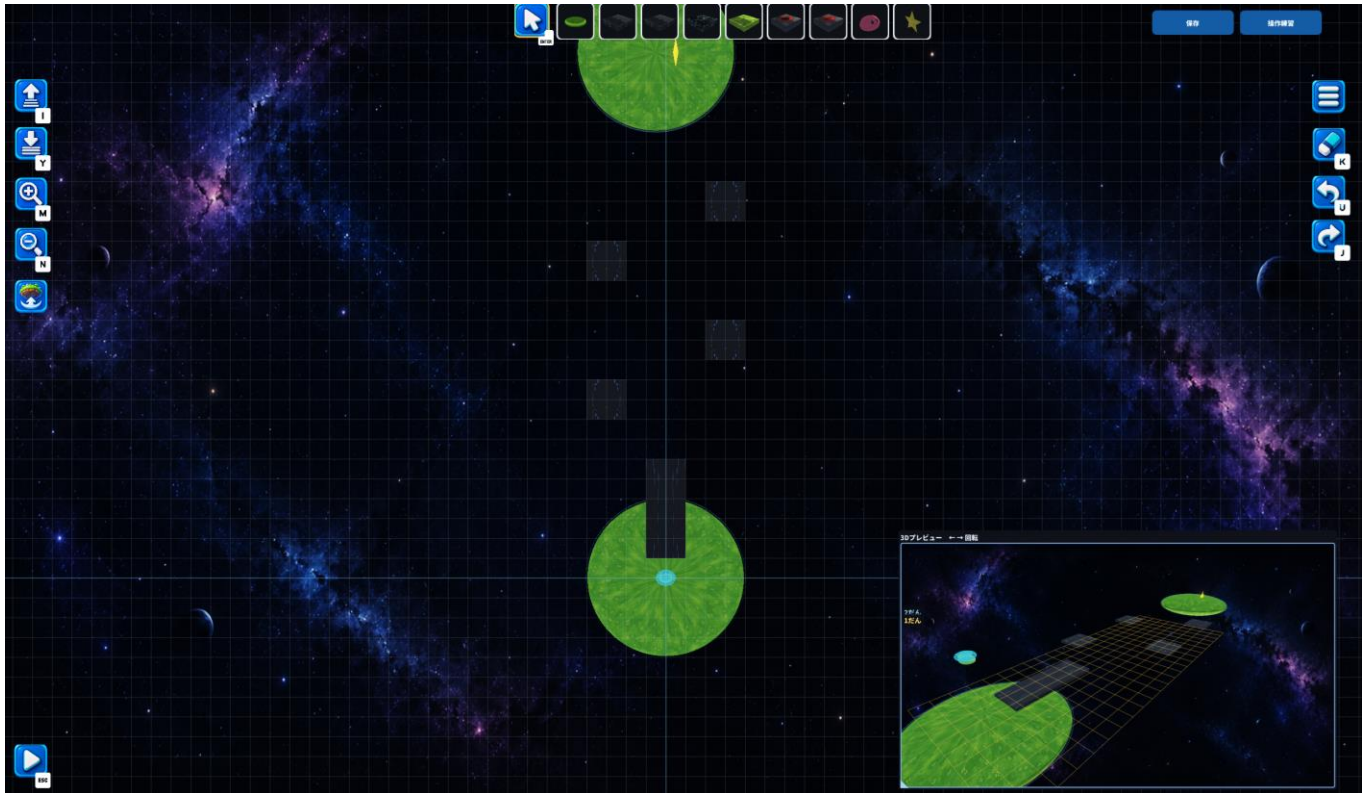
## 6. プレイヤー向けステージ作成機能

### 6-1. 「遊ぶ人」から「遊びを作る人」へ

昨年度作品には、プレイヤー自身がステージを作成する機能はありませんでした。本年度作品では、ゲームを遊ぶだけでなく自分でステージを作って楽しめるよう、プレイヤー向けのステージ作成機能を実装しました。

一般的な 3D エディタのように X・Y・Z 軸を自由に操作する方式は、3D 編集に慣れていない人には位置関係を把握しにくいと考えました。そこで水平方向を 2D グリッド、高さ方向を「だん」として分離し、マス単位の配置と高さの切り替えで立体的なステージを作成できる仕組みにしました。

編集中は 3D プレビューで見え方を確認でき、配置、Undo / Redo、保存、プレイテスト、クリア確認まで一連の流れを用意しています。内部では 3D 座標を扱いながら、プレイヤーには専門的な 3D 操作をできるだけ意識させず、積み木を組み立てるような感覚でステージ制作へ参加できることを目指しました。



2D 操作と 3D プレビューを組み合わせたプレイヤー向けステージ作成機能

## 7. 次回作でさらに実現したいこと

### 7-1. 本年度作品で残ったハードル

本年度作品では、操作の複雑さ、入力方法、言語、プレイ人数、ステージ制作など、遊びにくさにつながる要因を実装によって減らすことを意識しました。一方で、言語の違い、操作方法の好み、一緒に遊ぶ人の有無など、まだ対応できていない違いもあります。

次回作では、本年度に構築した入力、Player 制御、AI、UI・テキストなどの基盤を発展させ、さらに多くの人と同じ遊びへ参加できる仕組みを実装したいと考えています。

### 7-2. 複数言語へ対応するローカライズ

本年度作品では日本語表示とルビ表示へ対応しました。次回作では、UI テキストを言語ごとのデータとして管理し、フォントや文字幅、UI レイアウトの違いまで考慮して複数言語へ対応したいと考えています。日本語・ルビ対応で得た経験を、使用する言語によって遊びにくくならない仕組みへ発展させます。

### 7-3. 1 人でも協力して遊べる Companion NPC AI

本年度作品では、1 人プレイ時に 2 体を切り替え、2 人プレイ時にはそれぞれを別のプレイヤーが操作する仕組みを実装しました。

次回作では、この Player 制御や 2 人プレイの仕組みと、Enemy の行動 AI で得た知識を生かし、1 人プレイ時にはもう一方のキャラクターが自律して戦闘やギミック攻略を補助する仕組みへ発展させたいと考えています。一緒に遊ぶ人がいない場合でも、協力する遊びを楽しめることを目指します。

### 7-4. プレイヤーに合わせた入力・表示のカスタマイズ

本年度作品ではキーボードとゲームパッドの両方へ対応しましたが、操作方法や情報の見やすさを個人に合わせて調整する仕組みは十分ではありません。

次回作では、ボタン割り当て変更や文字サイズなどを設定できるようにし、単に複数の入力方法へ対応するだけでなく、自分に合った操作・表示を選べるゲームへ発展させたいと考えています。

## 8. まとめ

### 8-1. この 1 年間での成長

この 1 年間の成長は、より高度な技術を使えるようになったことだけではありません。固定機能中心の描画から Shader・3D モデル・アニメーションへ、Y 軸固定の AABB 中心の移動から Ray Test・Sweep Test を用いた 360°移動へ、数値編集とホットリロードから画面上で直接調整できるエディタへと、ゲームを支える技術基盤そのものを発展させました。

同時に、技術や機能を増やすだけでなく、それらをプレイヤー体験へどう結び付けるかを考えるようになりました。戦闘、2 人プレイ、日本語・ルビ、ステージ作成機能などを通して、遊びの幅を広げながら、その遊びへ参加するためのハードルはできるだけ上げないことを重視しました。

今後も、遊びを単純にすることで入口を下げるのではなく、**遊びの深さを保ったまま、それを楽しめる人を技術によって広げられるゲーム開発**を目指します。